

```

import numpy as np

#=====

# Problem 3

#=====

def f(x):
    y = np.exp(x)
    return y - 1
x = 9.999999995000000*10**(-10)
answer = f(x)
print (answer)

y = np.exp(x)
alg = y - 1
true = np.expm1(x)
rel_error = abs(alg - true)/abs(true)
digits = -np.log10(rel_error)
print("algorithm =", alg)
print("true =", true)
print("relative error:", rel_error)
print ("correct digits:", digits)

Taylor = x + x**2/2
rel_error2 = abs(Taylor - true)/abs(true)
digits2 = -np.log10(rel_error2)
print("Taylor = ", Taylor)

```

```
print("relative error =", rel_error2)

print("correct digits with Taylor series = ", digits2)
```

```
#=====
```

```
# Problem 4
```

```
#=====
```

```
def driver():
```

```
# use routines
```

```
    f = lambda x: 2*x - 1 - np.sin(x)
```

```
    a = 0
```

```
    b = 1
```

```
# f = lambda x: np.sin(x)
```

```
# a = 0.1
```

```
# b = np.pi+0.1
```

```
    tol = 5e-9
```

```
    [astar,ier] = bisection(f,a,b,tol)
```

```
    print('the approximate root is',astar)
```

```
    print('the error message reads:',ier)
```

```
    print('f(astar) =', f(astar))
```

```
# define routines
```

```
def bisection(f,a,b,tol):
```

```
# Inputs:
```

```
# f,a,b - function and endpoints of initial interval
```

```
# tol - bisection stops when interval length < tol
```

```
# Returns:
```

```
# astar - approximation of root
```

```
# ier - error message

# - ier = 1 => Failed

# - ier = 0 == success

# first verify there is a root we can find in the interval

fa = f(a)

fb = f(b);

if (fa*fb>0):

    ier = 1

    astar = a

    return [astar, ier]

# verify end points are not a root

if (fa == 0):

    astar = a

    ier = 0

    return [astar, ier]

if (fb == 0):

    astar = b

    ier = 0

    return [astar, ier]

count = 0

d = 0.5*(a+b)

while (abs(d-a)> tol):

    fd = f(d)

    if (fd == 0):

        astar = d

        ier = 0
```

```

        return [astar, ier]

    if (fa*fd<0):
        b = d
    else:
        a = d
        fa = fd
    d = 0.5*(a+b)
    count = count +1

# print('abs(d-a) = ', abs(d-a))

    astar = d
    ier = 0

    print('count = ', count)

    return [astar, ier]

driver()

#=====

# Problem 5

#=====


def driver1():

# use routines

    f = lambda x: x**3 + x - 4

    a = 1

    b = 4

# f = lambda x: np.sin(x)

# a = 0.1

```

```

# b = np.pi+0.1

tol = 1e-3

[astar,ier] = bisection(f,a,b,tol)

print('the approximate root is',astar)

print('the error message reads:',ier)

print('f(astar) =', f(astar))

# define routines

def bisection(f,a,b,tol):

# Inputs:

# f,a,b - function and endpoints of initial interval

# tol - bisection stops when interval length < tol

# Returns:

# astar - approximation of root

# ier - error message

# - ier = 1 => Failed

# - ier = 0 == success

# first verify there is a root we can find in the interval

fa = f(a)

fb = f(b);

if (fa*fb>0):

    ier = 1

    astar = a

    return [astar, ier]

# verify end points are not a root

if (fa == 0):

    astar = a

```

```
    ier = 0

    return [astar, ier]

if (fb == 0):

    astar = b

    ier = 0

    return [astar, ier]

count = 0

d = 0.5*(a+b)

while (abs(d-a) > tol):

    fd = f(d)

    if (fd == 0):

        astar = d

        ier = 0

        return [astar, ier]

    if (fa*fd < 0):

        b = d

    else:

        a = d

        fa = fd

    d = 0.5*(a+b)

    count = count + 1

# print('abs(d-a) = ', abs(d-a))

astar = d

ier = 0

print('count= ', count)

return [astar, ier]
```

```
driver1()
```

```
#=====
```

```
# Problem 6
```

```
#=====
```

```
import matplotlib.pyplot as plt
```

```
x = np.linspace(-10,10,10000)
```

```
y = x - 4*np.sin(2*x) - 3
```

```
plt.plot(x,y)
```

```
plt.xlabel("x")
```

```
plt.ylabel("f(x)")
```

```
plt.title("f(x) = x - 4sin(2x) - 3")
```

```
plt.axhline(0)
```

```
plt.show()
```

```
def driver2():
```

```
# test functions
```

```
    f1 = lambda x: -np.sin(2*x) + 5*x/4 - 3/4
```

```
    gp = lambda x: -2*np.cos(2*x) + 5/4
```

```
    Nmax = 100
```

```
    tol = 1e-11
```

```
# test f1 '''
```

```
    init_guess = np.linspace(-1.5,7.5,10)
```

```
for x0 in init_guess:
    [xstar,ier,guesses] = fixedpt(f1,x0,tol,Nmax)
```

```
if ier == 0:
    print("|g'(x)| = ", abs(gp(xstar)))
    print("initial guess:", x0)
    print('the approximate root is:',xstar)
    print('f1(xstar):',f1(xstar))
    print('Error message reads:',ier)
    print()
```

```
# define routines
```

```
def fixedpt(f,x0,tol,Nmax):
```

```
# "" x0 = initial guess""
```

```
# "" Nmax = max number of iterations""
```

```
# "" tol = stopping tolerance""
```

```
count = 0
```

```
guesses = []
```

```
while (count < Nmax):
```

```
    count = count + 1
```

```
    x1 = f(x0)
```

```
guesses.append(x1)
if (abs(x1-x0) < tol):
    xstar = x1
    ier = 0
    return [xstar,ier,np.array(guesses)]
x0 = x1
```

```
xstar = x1
ier = 1
return [xstar,ier,np.array(guesses)]
```

```
driver2()
```